



Projet de Fin d'Études:
PICO-CENTRALE HYDROÉLECTRIQUE
en vue de l'obtention du
Brevet de Technicien Supérieur de
Cybersécurité ,Informatique et Réseaux ,Électronique
Spécialité : Électronique et Réseau

PARTIE 1: Mesure hauteur d'eau
Etudiant 1: Jessy SACLIER

Contexte

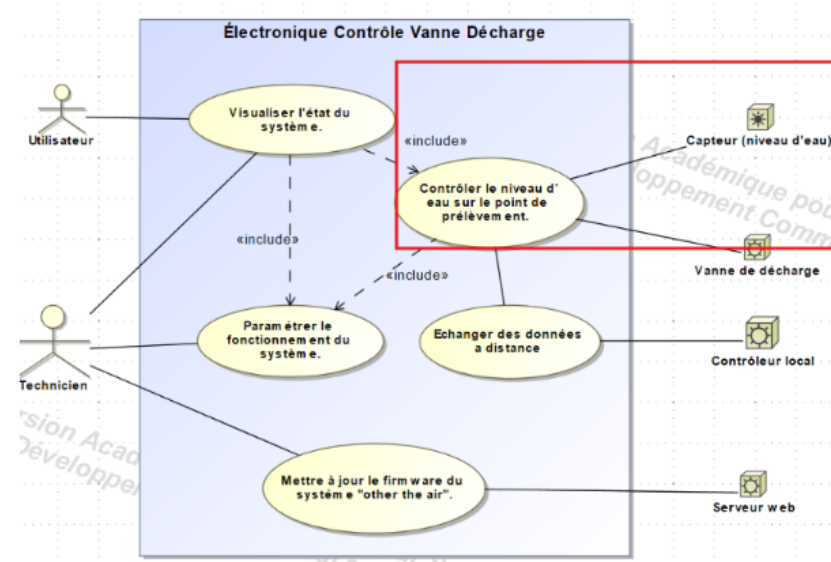
- Ma mission:**
- Garantir fiabilité mesure du niveau d'eau
 - Testé et sélectionné le capteur le plus adapté
 - Développé programme sur microcontrôleur ESP32
 - Filtrer erreurs de lecture et exploiter données

Tâches

Bloc ID = 1.1.1

Cas d'utilisation

«requirement»
Mesure du niveau d'eau au point de prélèvement.
Id = "1.1.1"
Text = "La mesure sera réalisée à l'aide d'un capteur de type Lidar capable de mesurer des distances au centimètre près. Il sera interfacé sur bus I2C. Le capteur sera positionné à moins d'un mètre du processeur. Une mesure toutes les 1 à 3 secondes sera suffisante."



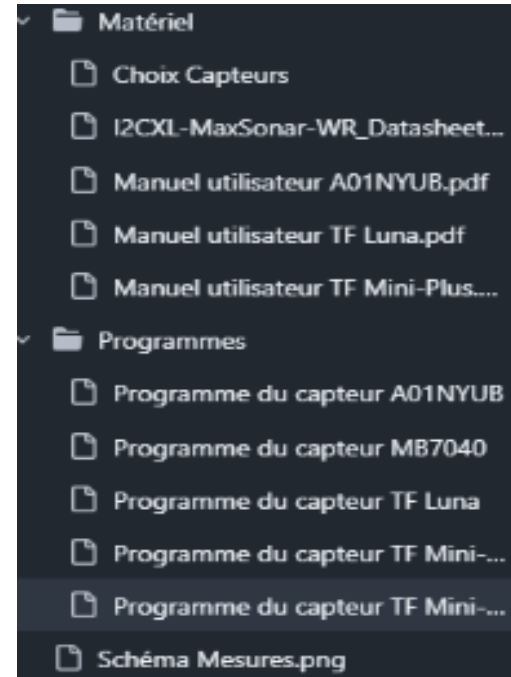
Organisation

Plateforme collaborative: GitHub

Branches:



Fichiers:



Principe

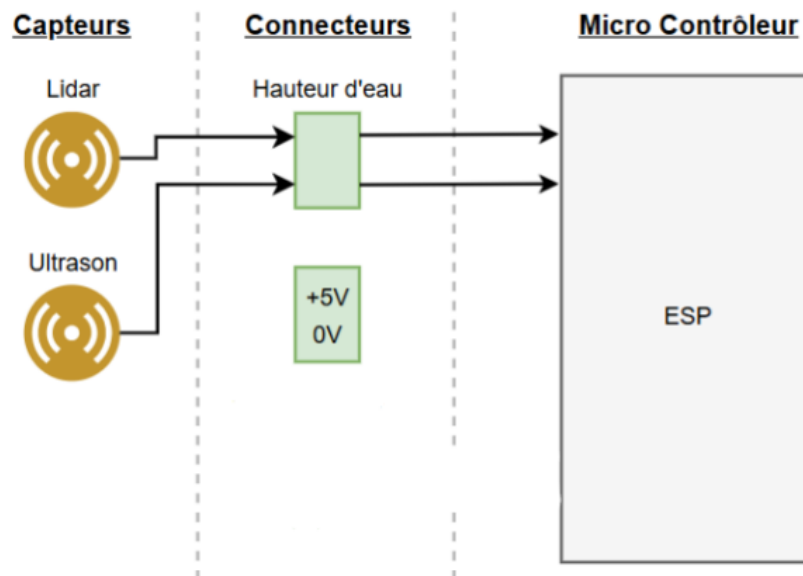
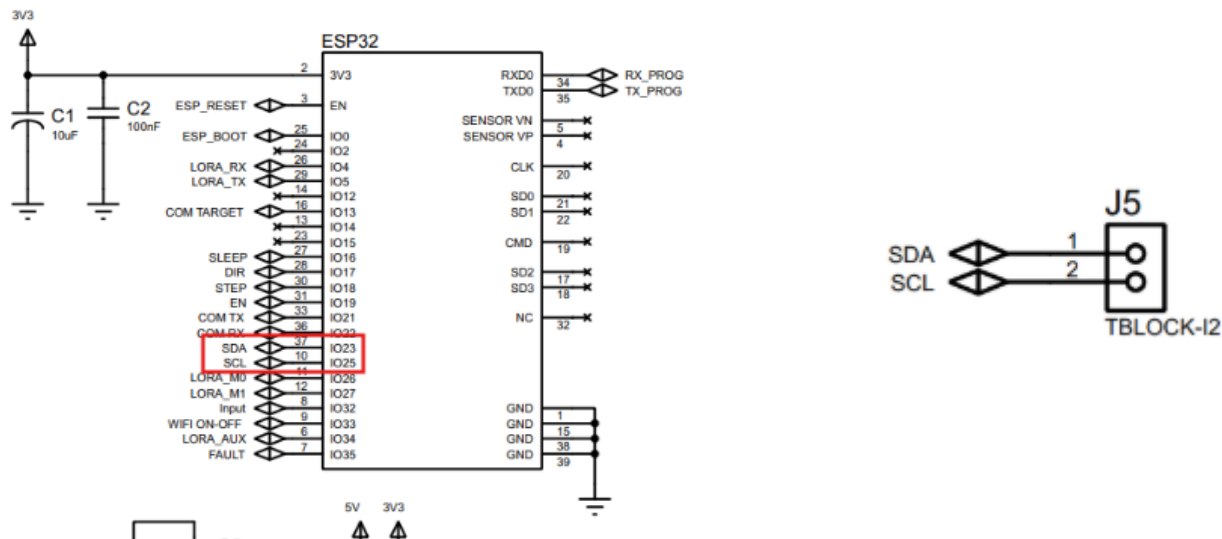
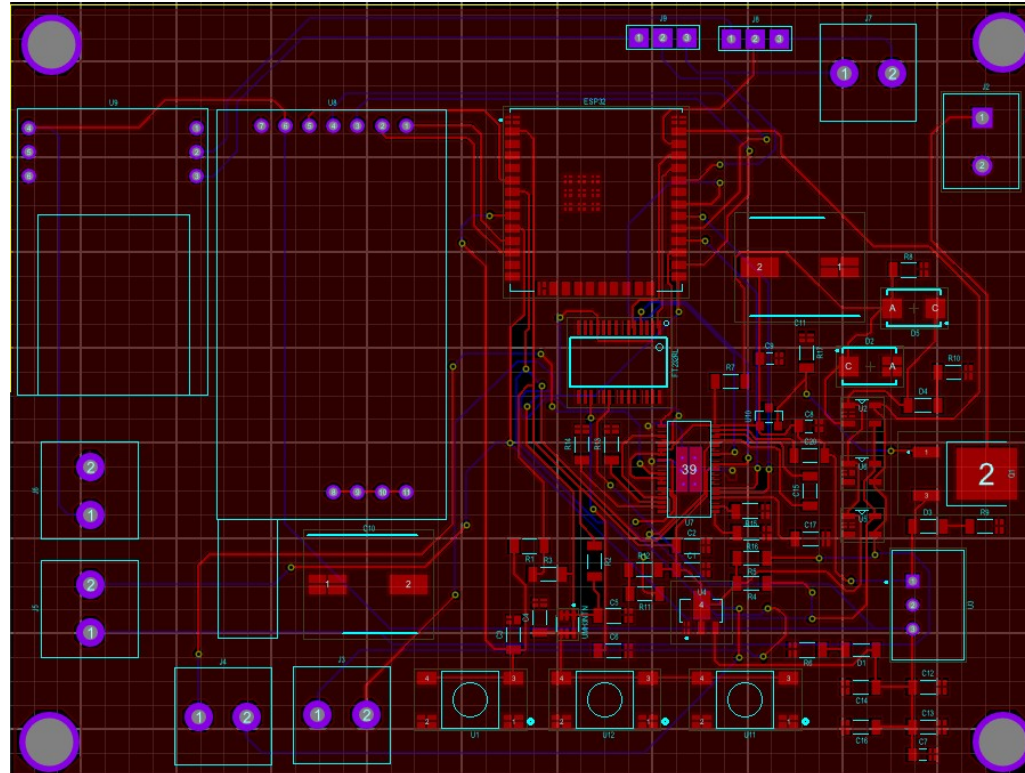


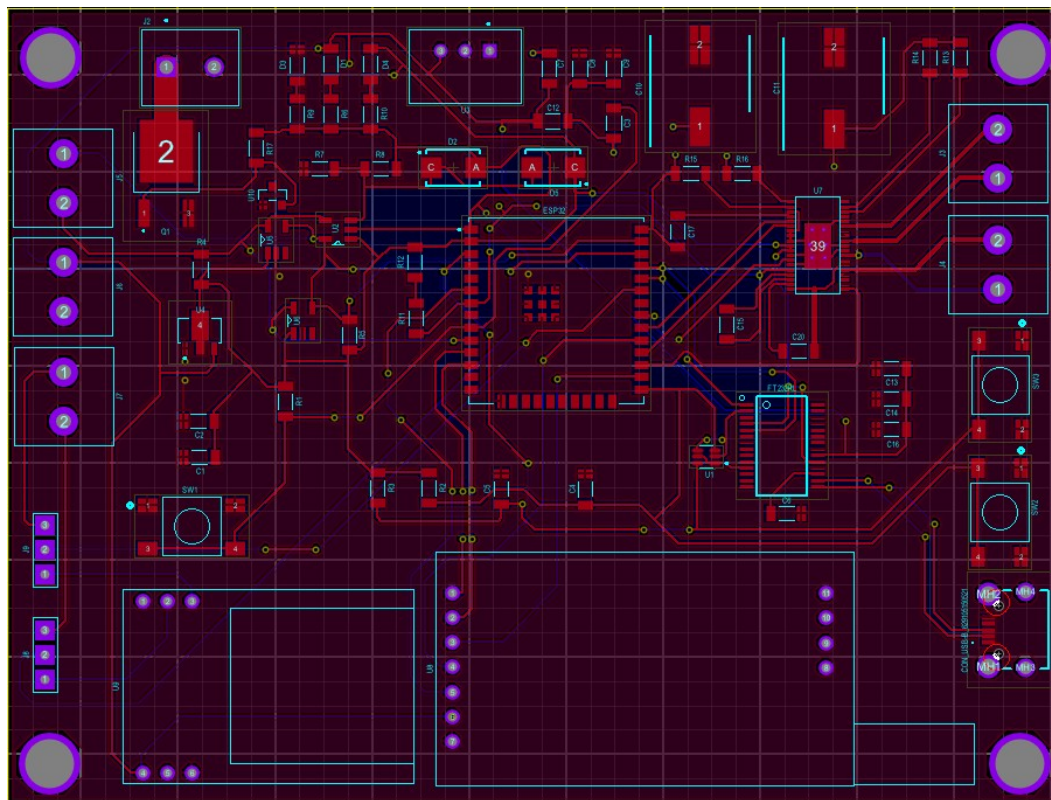
Schéma électrique



Routage



Routage



Capteurs

Capteur TF Mini-Plus



- Distance de mesure:** 0,1m -> 12m
- Technologie:** Lidar
- Résolution:** 1cm
- Rafraîchissement:** 1~1000Hz
- Prix:** 60€

Capteurs

Capteur TF Luna



- Distance de mesure:** 0,2m -> 8m
- Technologie:** Lidar
- Résolution:** 1cm
- Rafraîchissement:** 1~250Hz
- Prix:** 25€

Capteurs

Capteur A01NYUB



- Distance de mesure:** 0,28m -> 7,5m
- Technologie:** Ultrason
- Résolution:** 1mm
- Rafraîchissement:** ~10Hz
- Prix:** 40€

Capteurs

Capteur MB7040

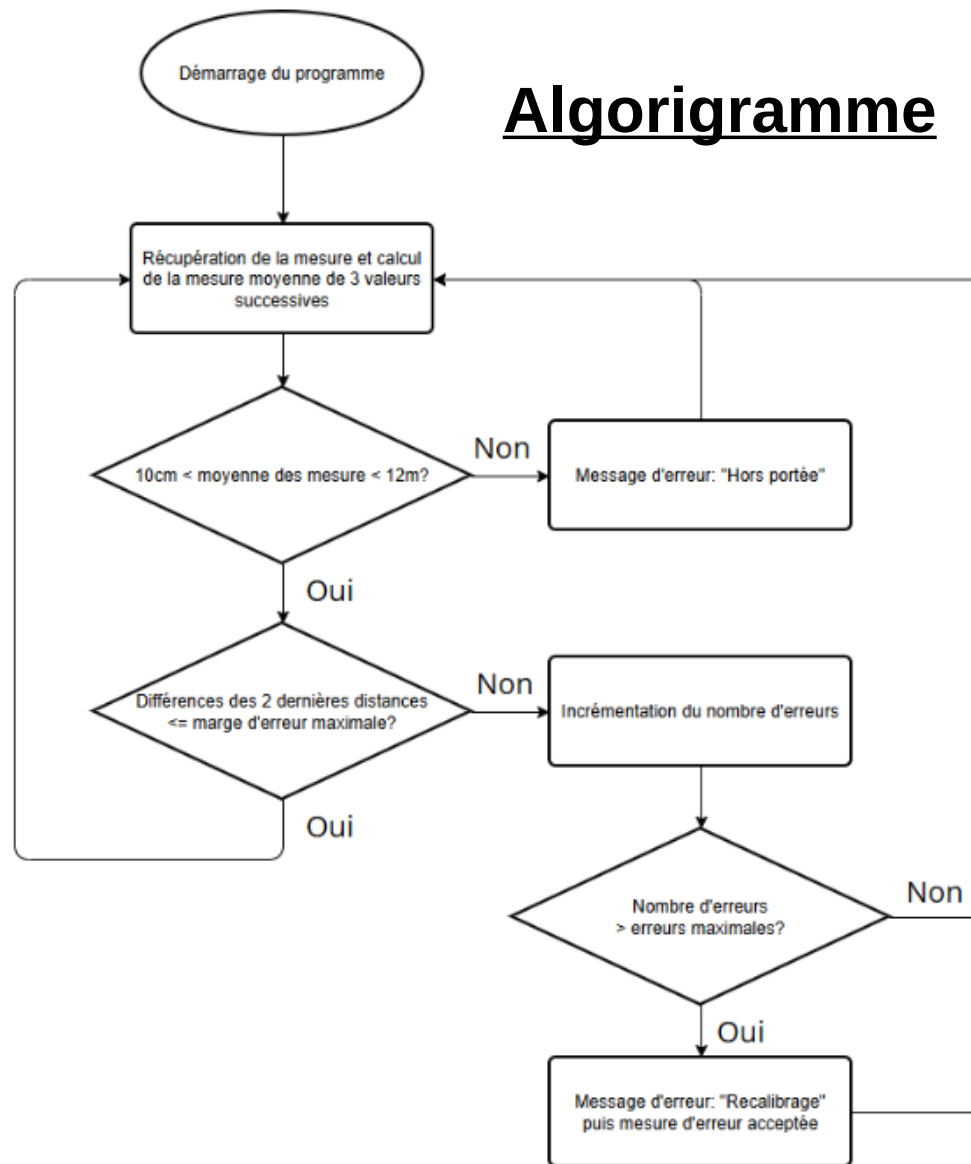


- Distance de mesure:** 0,2m -> 7,65m
- Technologie:** Ultrason
- Résolution:** 1cm
- Rafraîchissement:** ~10Hz
- Prix:** 95€

Matrice de choix

	Capteurs	TF Mini-Plus	TF Luna	A01NYUB	MB7040
Critères					
Distance de mesure /2		2	1	1	1
Technologie /2		2	2	1	1
Résolution /2		2	2	2	2
Prix /2		1	2	2	1
Étanchéité / 2		2	0	2	2
Total /10		9	7	8	7

Algorithme



Programme

```
if devices:
    for d in devices:
        version = get_firmware_version(d)
        sensor_info[d] = version
        print("Trouvé : Adresse {} | Version : {}".format(hex(d), version))

    # Première adresse trouvée = capteur principal
    TFMINI_ADDR = devices[0]
else:
    # Echec scan = adresse par défaut (0x10 pour le TFMini Plus en mode I2C)
    print("\nAucun capteur détecté. Utilisation adresse par défaut 0x10")
    TFMINI_ADDR = 0x10
```

```
dist1, dist2, dist3, moyenne = 0, 0, 0, 0 # Stockage des 3 dernières valeurs et de la moyenne
error, count = 0, 0 # Valeurs des erreurs et du compteur
max_error, max_marge = 2, 10 # Nombre d'erreurs tolérées et variation maximale autorisée (en cm) entre deux mesures
timer_period_ms, error_period_ms = 3000, 1000 # Fréquence de mesure normale / erreur détectée
att_moy = 100 # Attente en ms entre chaque distance mesurée
```

Programme

```
try:
    # Commande demande de mesure de distance
    i2c.writeto(TFMINI_ADDR, b'\x5a\x05\x00\x01\x60')

    # Lecture 9 octets de données de mesure
    data = i2c.readfrom(TFMINI_ADDR, 9)

    # Vérification protocole standard TFMini
    if len(data) >= 9 and data[0] == 0x59 and data[1] == 0x59:
        # Distance codée sur 2 octets
        # Octet 2 (poids faible) + octet 3 (poids fort) décalé de 8 bits vers la gauche
        distance = data[2] + (data[3] << 8)
        distance = round(distance * 1.05)
        return distance
    except:
        print("\nErreur dans la récupération de la mesure.")
        return None

return None
```

Programme

```
# Prise de mesure décalée pour calculer une moyenne
dist3 = mesure

time.sleep_ms(att_moy)
mesure = get_distance()
dist2 = mesure

time.sleep_ms(att_moy)
mesure = get_distance()
dist1 = mesure

moyenne = round((dist1 + dist2 + dist3)/3)

# Variance trop élevée = ignorée + erreur + temps de mesure raccourci
timer_sensor.init(mode=Timer.PERIODIC, period=error_period_ms, callback=get_data)
error += 1
print(f"{error} erreur(s)\n")

if error > max_error:
    # Erreurs maximales atteintes = réinitialisation des variables + reprise du cycle normal
    print("Erreurs multiples, recalibrage...\n\n")
    error = 0
    dist1 = mesure
    dist2 = mesure
    dist3 = mesure
    timer_sensor.init(mode=Timer.PERIODIC, period=timer_period_ms, callback=get_data)
```

Affichage

Capteur A01NYUB

```
Distance n°1: 0 cm  
Moyenne (3 valeurs): 0 cm | Erreur: 1  
Octets: [0000000000000000]
```

```
Distance n°2: 0 cm  
Moyenne (3 valeurs): 0 cm | Erreur: 2  
Octets: [0000000000000000]
```

```
--- Ajustement suite à 3 erreurs ---
```

```
Distance n°3: 192 cm  
Moyenne (3 valeurs): 64 cm | Erreur: 0  
Octets: [0000000011000000]
```

```
Distance n°4: 192 cm  
Moyenne (3 valeurs): 128 cm | Erreur: 0  
Octets: [0000000011000000]
```

```
Distance n°5: 192 cm  
Moyenne (3 valeurs): 192 cm | Erreur: 0  
Octets: [0000000011000000]
```

-Ajustement après erreurs

-Nombre mesure

-Distance moyenne

-Mesure en décimal et binaire

Affichage

Capteur TF Mini-Plus

```
Distance n°1: 0 cm  
Moyenne des 3 dernières valeurs: 0 cm | Erreur: 1  
  
Distance n°2: 0 cm  
Moyenne des 3 dernières valeurs: 0 cm | Erreur: 2  
  
--- Ajustement suite à erreurs répétées ---  
  
Distance n°3: 231 cm  
Moyenne des 3 dernières valeurs: 77 cm | Erreur: 0  
  
Distance n°4: 231 cm  
Moyenne des 3 dernières valeurs: 154 cm | Erreur: 0  
  
Distance n°5: 232 cm  
Moyenne des 3 dernières valeurs: 231 cm | Erreur: 0  
  
Distance n°6: 231 cm  
Moyenne des 3 dernières valeurs: 231 cm | Erreur: 0  
  
Distance n°7: 231 cm  
Moyenne des 3 dernières valeurs: 231 cm | Erreur: 0
```

-Ajustement après erreurs

-Nombre mesure

-Distance moyenne

Affichage

Capteur TF Mini-Plus

```
Recherche de capteurs I2C...  
Trouvé : Adresse 0x10 | Version : V5.1.2  
[Adresse: 0x10 | Version: V5.1.2]  
Distance n°1: 248 cm  
Format bytes: b'\xf8\x00'
```

```
1 erreur(s)
```

```
2 erreur(s)
```

```
3 erreur(s)
```

```
Erreurs multiples, recalibrage...
```

```
[Adresse: 0x10 | Version: V5.1.2]  
Distance n°2: 16 cm  
Format bytes: b'\x10\x00'
```

```
[Adresse: 0x10 | Version: V5.1.2]  
Distance n°3: 16 cm  
Format bytes: b'\x10\x00'
```

-Ajustement après erreurs

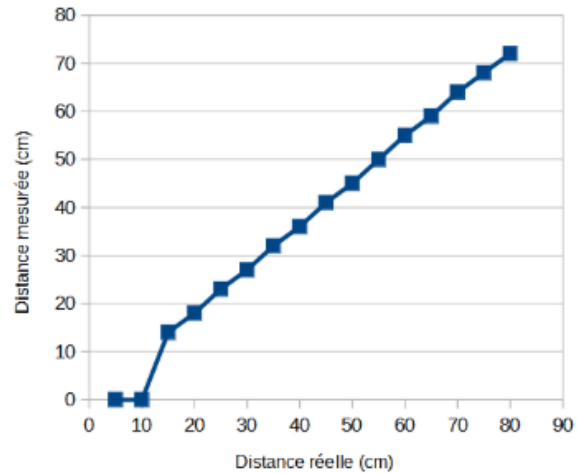
-Nombre mesure

-Adresse

-Distance moyenne

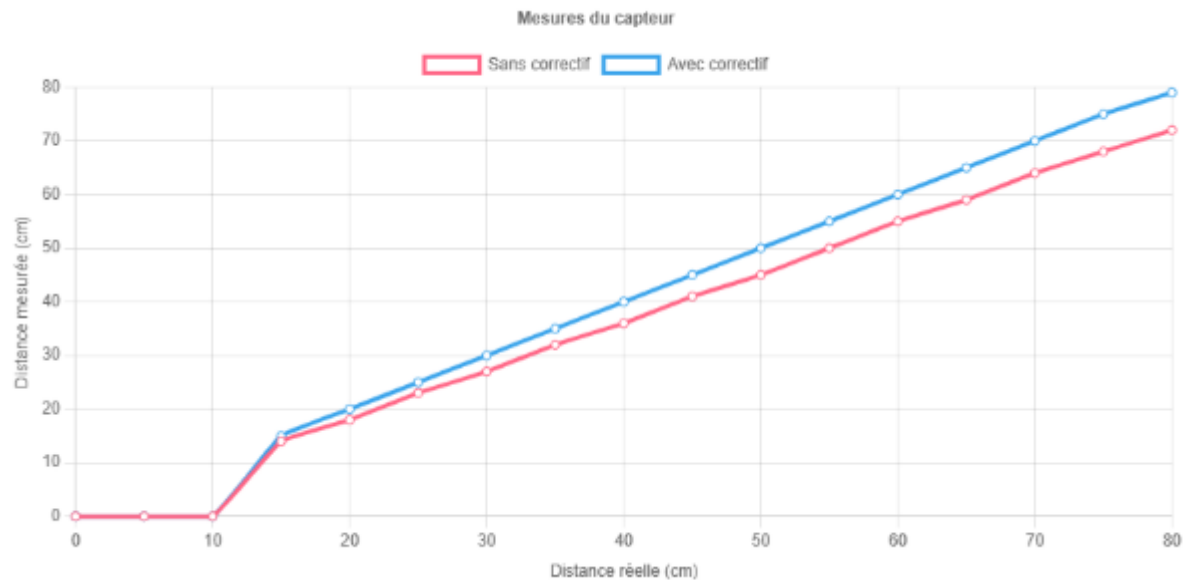
-Mesure en décimal et hex

Mesures



Distance réelle (cm)	Distance mesurée (cm)	Différence réelle/mesurée (cm)
5	0	Hors Portée
10	0	Hors Portée
15	14	1
20	18	2
25	23	2
30	27	3
35	32	3
40	36	4
45	41	4
50	45	5
55	50	5
60	55	5
65	59	6
70	64	6
75	68	7
80	72	8

Mesures



Trame de données



Obtain Data Frame	5A 05 00 01 60	Data Frame(9bytes- cm)	Only works in IIC mode	/
	5A 05 00 06 65	Date Frame(9bytes- mm)		

Trame de données



Byte0 -1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7	Byte8
0x59 59	Dist_L	Dist_H	Strength_L	Strength_H	Temp_L	Temp_H	Checksum
Data code explanation							
Byte0	0x59, frame header, same for each frame						
Byte1	0x59, frame header, same for each frame						
Byte2	Dist_L distance value low 8 bits						
Byte3	Dist_H distance value high 8 bits						
Byte4	Strength_L low 8 bits						
Byte5	Strength_H high 8 bits						
Byte6	Temp_L low 8 bits						
Byte7	Temp_H high 8 bits						
Byte8	Checksum is the lower 8 bits of the cumulative sum of the numbers of the first 8 bytes.						

Conclusion